

yovel\QP1\hw7_4.py

```

1
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from matplotlib.animation import FuncAnimation
5
6 # ***** (2.1) *****
7
8 def integrate_riemann(f):
9     dy = 20.0 / (len(f) - 1)
10    return np.sum(f) * dy
11
12 # ***** (2.2) *****
13
14 print("\nThe first steps of the exercise")
15 print("*****")
16
17 h_bar = 1.054571817e-34
18
19 def phi_0(p):
20     N = 1
21     L = 1.0 # [cm]
22
23     p0 = h_bar / L
24     delta = h_bar / L
25     x0 = -5 * L
26
27     gaussian = np.exp(-((p - p0) ** 2) / (2 * (delta ** 2)))
28
29     phase = np.exp(-1j * p * x0 / h_bar)
30
31     return N * gaussian * phase
32
33 def phi(x, t):
34     L = 1.0 # [cm]
35     p0 = h_bar / L
36     m = h_bar / L
37
38     y = np.linspace(-10, 10, 2000)
39
40     p = y * p0
41
42     free = (p ** 2) / (2 * m)
43     exponent = (1j / h_bar) * (p * x - free * t)
44
45     integrand = phi_0(p) * np.exp(exponent)
46
47     integral_y = integrate_riemann(integrand)
48
49     integral_p = integral_y * p0
50
51     normalization_factor = 1 / np.sqrt(2 * np.pi * h_bar)

```

```

52
53     return normalization_factor * integral_p
54
55 print(f" $\psi(0, 0) = \{\phi(0, 0)\}$ ")
56
57 # ***** (2.3) *****
58
59 x = np.arange(-10, 10.1, 0.1)
60
61 psi_unnormalized = np.array([phi(x, 0) for x in x])
62
63 # ***** (2.4) *****
64
65 probability_density = np.abs(psi_unnormalized) ** 2
66
67 integral_x = integrate_riemann(probability_density)
68
69 N = 1 / np.sqrt(integral_x)
70
71 print(f"N = {N}")
72
73 # ***** (2.5) *****
74
75 x_vec = np.arange(-10, 10.1, 0.1)
76 t_vec = np.arange(0, 10.1, 0.1)
77
78 # empty matrix, rows will represent time steps, columns will represent spatial points.
79 psi_xt = np.zeros((len(t_vec), len(x_vec)), dtype=complex)
80
81 for i, t in enumerate(t_vec):
82     for j, x in enumerate(x_vec):
83         psi_xt[i, j] = N * phi(x, t)
84
85 # ***** (2.6) *****
86
87 prob_density = np.abs(psi_xt)**2
88
89 fig, ax = plt.subplots(figsize=(8, 5))
90
91 ax.set_xlim(-10, 10)
92 ax.set_ylim(0, np.max(prob_density) * 1.1)
93
94 ax.set_title('Time Evolution of Probability Density  $|\psi(x,t)|^2$ ')
95 ax.set_xlabel('Position (x)')
96 ax.set_ylabel('Probability Density')
97 ax.grid(True)
98
99 line, = ax.plot([], [], lw=2, color='blue')
100
101 # required for this graphic library
102 def init():
103     line.set_data([], [])
104     return line,
105

```

```

106 # required for this graphic librery
107 def animate(i):
108     line.set_data(x_vec, prob_density[i, :])
109     ax.set_title(f'Time Evolution ($t = {t_vec[i]:.1f}$)')
110     return line,
111
112 # main animation engine
113 anim = FuncAnimation(fig, animate, init_func=init,
114                     frames=len(t_vec), interval=100, blit=True)
115
116 # save to GIF
117 anim.save('part2_6.gif', writer='pillow', fps=10)
118
119 # generate the animation
120 plt.show()
121
122 # ***** (2.7) *****
123
124 print("\npart 2.7:")
125 print("*****")
126
127 h_bar = 1.054571817e-34
128
129 # The Fourier transform of the given function which I assume should be:  $\psi(x, 0)$ 
130 def phi_0_zayin(p):
131     N = 1
132     L = 1.0
133
134     p0 = h_bar / L
135     delta = L
136     x0 = -5.0 * L
137
138     lorentz = np.exp(- (np.abs(p - p0) * delta) / h_bar)
139
140     phase = np.exp(-1j * p * x0 / h_bar)
141
142     return N * lorentz * phase
143
144 # The rest of the code is the same as the previous parts of the question...
145
146 def phi(x, t):
147     L = 1.0 # [cm]
148     p0 = h_bar / L
149     m = h_bar / L
150
151     y = np.linspace(-10, 10, 2000)
152
153     p = y * p0
154
155     free = (p ** 2) / (2 * m)
156     exponent = (1j / h_bar) * (p * x - free * t)
157
158     integrand = phi_0_zayin(p) * np.exp(exponent)
159

```

```

160     integral_y = integrate_riemann(integrand)
161
162     integral_p = integral_y * p0
163
164     normalization_factor = 1 / np.sqrt(2 * np.pi * h_bar)
165
166     return normalization_factor * integral_p
167
168     print(f" $\psi(0, 0) = \{\phi(0, 0)\}$ ")
169
170     x = np.arange(-10, 10.1, 0.1)
171
172     psi_unnormalized = np.array([phi(x, 0) for x in x])
173
174     probability_density = np.abs(psi_unnormalized) ** 2
175
176     integral_x = integrate_riemann(probability_density)
177
178     N = 1 / np.sqrt(integral_x)
179
180     print(f"N = {N}")
181
182     x_vec = np.arange(-10, 10.1, 0.1)
183     t_vec = np.arange(0, 10.1, 0.1)
184
185     # empty matrix, rows will represent time steps, columns will represent spatial points.
186     psi_xt = np.zeros((len(t_vec), len(x_vec)), dtype=complex)
187
188     for i, t in enumerate(t_vec):
189         for j, x in enumerate(x_vec):
190             psi_xt[i, j] = N * phi(x, t)
191
192     prob_density = np.abs(psi_xt)**2
193
194     fig, ax = plt.subplots(figsize=(8, 5))
195
196     ax.set_xlim(-10, 10)
197     ax.set_ylim(0, np.max(prob_density) * 1.1)
198
199     ax.set_title('Time Evolution of Probability Density  $|\psi(x,t)|^2$ ')
200     ax.set_xlabel('Position (x)')
201     ax.set_ylabel('Probability Density')
202     ax.grid(True)
203
204     line, = ax.plot([], [], lw=2, color='blue')
205
206     # required for this graphic library
207     def init():
208         line.set_data([], [])
209         return line,
210
211     # required for this graphic library
212     def animate(i):
213         line.set_data(x_vec, prob_density[i, :])

```

```

214     ax.set_title(f'Time Evolution ($t = \{t\_vec[i]:.1f\}$)')
215     return line,
216
217 # main animation engine
218 anim = FuncAnimation(fig, animate, init_func=init,
219                     frames=len(t_vec), interval=100, blit=True)
220
221 # save to GIF
222 anim.save('part2_7.gif', writer='pillow', fps=10)
223
224 # generate the animation
225 plt.show()
226
227 # ***** part 3 *****
228
229 print("\npart 3:")
230 print("*****")
231
232 h_bar = 1.0
233 m = 1.0
234 V0 = 5.0
235 x0 = -15.0
236 Delta_q = 0.5
237
238 # (E = V0)
239 q_critical = np.sqrt(2 * m * V0)
240
241 # Here you decide which section to run!
242 # =====
243 # a) (E < V0): q0 = 2.0
244 # b) (E > V0): q0 = 4.0
245 # c) (E ~ V0): q0 = q_critical
246 # =====
247
248 part_number = 1
249 parts = [2.0, 4.0, q_critical]
250
251 for part in parts:
252     q0 = part
253
254     def phi_step(x, t):
255         q_values = np.linspace(q0 - 6*Delta_q, q0 + 6*Delta_q, 1000)
256         dq = q_values[1] - q_values[0]
257
258         phi_0_q = np.exp(-((q_values - q0)**2) / (2 * Delta_q**2)) * np.exp(-1j * q_values
259 * x0 / h_bar)
260
261         E_q = (q_values**2) / (2 * m)
262
263         time_phase = np.exp(-1j * E_q * t / h_bar)
264
265         integral_sum = 0
266
267         for i, q in enumerate(q_values):

```

```

267     k_prime = np.sqrt(q**2 - 2 * m * V0 + 0j)
268
269     R = (q - k_prime) / (q + k_prime)
270     T = 2 * q / (q + k_prime)
271
272     if x < 0:
273         psi_xq = np.exp(1j * q * x / h_bar) + R * np.exp(-1j * q * x / h_bar)
274     else:
275         psi_xq = T * np.exp(1j * k_prime * x / h_bar)
276
277     integral_sum += phi_0_q[i] * psi_xq * time_phase[i] * dq
278
279     return integral_sum
280
281     # From here it's similar to the previous code... just creating the animation
282
283     print(f"Critical Momentum for E=V0 is: {q_critical:.2f}")
284     print("Calculating space-time grid. This will take a moment...")
285
286     x_values = np.arange(-30, 20, 0.2)
287     t_values = np.arange(0, 15, 0.1)
288
289     psi_xt = np.zeros((len(t_values), len(x_values)), dtype=complex)
290
291     for i, t in enumerate(t_values):
292         for j, x in enumerate(x_values):
293             psi_xt[i, j] = phi_step(x, t)
294
295     prob_density = np.abs(psi_xt)**2
296     prob_density = prob_density / np.max(prob_density)
297
298     fig, ax = plt.subplots(figsize=(10, 5))
299     ax.set_xlim(-30, 20)
300     ax.set_ylim(0, 1.1)
301
302     # We will see the potential step as a red line
303     ax.plot([0, 0], [0, 1.1], color='red', linestyle='--', alpha=0.5, label='Potential
Step (x=0)')
304     ax.fill_between([0, 20], 0, 1.1, color='red', alpha=0.05)
305
306     line, = ax.plot([], [], lw=2, color='blue', label='$|\psi(x,t)|^2$')
307     ax.legend(loc='upper right')
308     ax.set_xlabel('Position (x)')
309     ax.set_ylabel('Normalized Probability')
310
311     def init():
312         line.set_data([], [])
313         return line,
314
315     def animate(i):
316         line.set_data(x_values, prob_density[i, :])
317         ax.set_title(f'Wavepacket Scattering ($E_0$ proxy $q_0={q0}$) | Time:
{t_values[i]:.1f}')
318         return line,

```

```
319
320     anim = FuncAnimation(fig, animate, init_func=init,
321                           frames=len(t_values), interval=100, blit=True)
322
323     # save to GIF
324     anim.save(f'part3_{part_number}.gif', writer='pillow', fps=10)
325     part_number += 1
326
327     plt.show()
```